

INDIAN INSTITUTE OF TECHNOLOGY GANDHINAGAR

Department of Computer Science & Engineering | Semester V

Kubernetes Network Self-Healing Operator

Automated Detection, Closed-Loop Remediation, and Observability for CNI Crashes, CoreDNS Degradation, and NetworkPolicy Failures

Course:	Computer Networks (CN)
Project Type:	Custom Kubernetes Operator & Chaos Engineering
Tech Stack:	Go (client-go), Kubernetes (KIND), Calico/Flannel, Prometheus, Alertmanager, Grafana, Loki
Branch:	gobuild (Integrated Production Release)
Date:	September 2026

1. Executive Summary & Problem Statement

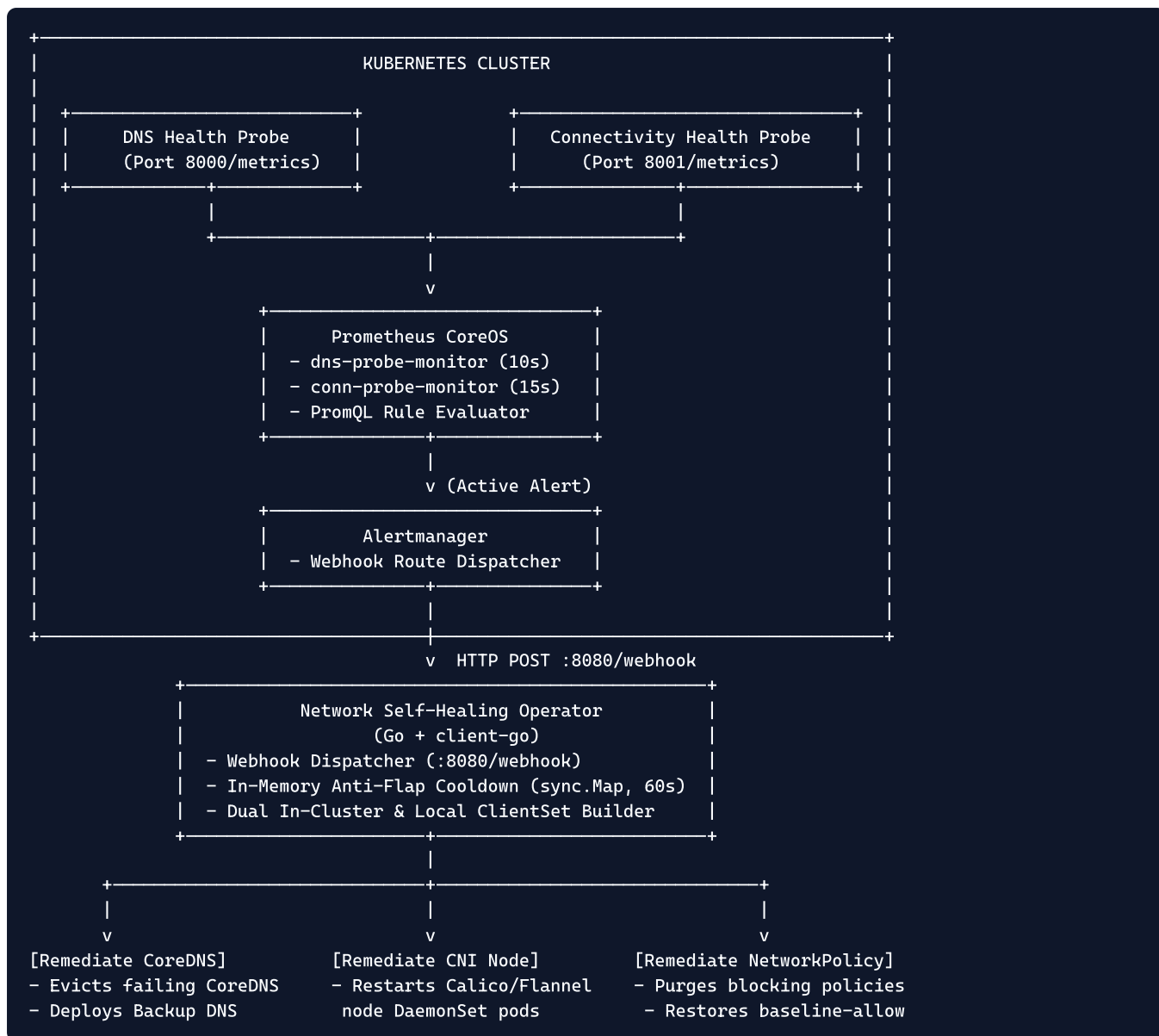
Kubernetes provides robust, built-in self-healing primitives for compute workloads, such as restarting failing application containers via kubelet liveness probes and rescheduling pods when compute nodes experience hardware failure. However, **Kubernetes fundamentally lacks built-in automated recovery for core networking failures.**

When cluster networking degrades—such as CNI daemonset crashes, CoreDNS latency degradation, or accidental misconfigurations of Kubernetes `NetworkPolicy` resources that block inter-service traffic—standard Kubernetes controllers cannot diagnose or remediate the root cause. This leads to silent outages, cascade failures, and application downtime that typically requires manual operator intervention.

To solve this problem, we developed a **production-grade, closed-loop Kubernetes Network Self-Healing Operator** in Go (`client-go`) paired with an active probing and Prometheus/Alertmanager monitoring pipeline. The system monitors cluster network health, detects faults via metrics, routes alerts through Alertmanager webhooks, and executes automated remediation actions within seconds.

2. Architecture & Closed-Loop Design

The architecture follows a decoupled, event-driven closed-loop reconciliation cycle comprising four tiers: Active Probes, Observability Pipeline, Self-Healing Operator Engine, and Automated Remediation Actions.



3. Core Components & Implementation Details

3.1 Active In-Cluster Probes (`probes/`)

Unlike passive log scrapers, our system deploys continuous active network probes packaged in lightweight containers (`python:3.10-slim` with `kubectl` and `prometheus-client`) in the `monitoring` namespace:

- **DNS Health Probe (`dns_probe.py`):** Measures query latency against `kubernetes.default.svc.cluster.local` and queries CoreDNS pod phases every 10 seconds. Exports `dns_latency_ms` (Gauge) and `dns_healthy` (Binary Gauge: 1 = healthy, 0 = unhealthy) on HTTP port 8000.
- **Cross-Namespace Connectivity Probe (`connectivity_probe.py`):** Executes automated remote curl checks between client pods across namespaces (`test-namespace-2` to `test-namespace-1`). Uses `curl -s -o /dev/null -w "%{http_code}"` to suppress body overhead and accurately detect network partitions. Exports `connectivity_success` and `connectivity_latency_ms` on HTTP port 8001.
- **ServiceMonitors (`manifests/monitoring/service-monitors.yaml`):** Prometheus Operator CRDs targeting the probe Services at intervals of 10s and 15s.

3.2 Prometheus Alerting & Alertmanager Routing (`manifests/monitoring/`)

Alert rules are codified as declarative `PrometheusRule` objects:

Alert Name	PromQL Expression	Evaluation Window	Severity
<code>DNSResolutionFailed</code>	<code>dns_healthy == 0</code>	15 seconds	Critical
<code>DNSLatencyHigh</code>	<code>dns_latency_ms > 100</code>	30 seconds	Warning
<code>PodConnectivityBlocked</code>	<code>connectivity_success == 0</code>	15 seconds	Critical
<code>CNIPodCrash</code>	<code>kube_pod_status_phase{phase="Failed"} > 0 or increase(kube_pod_container_status_restarts_total[5m]) > 0</code>	15 seconds	Critical

Scrutinizer Optimization: Rate-Increase Window

Standard raw counter thresholds (e.g., `restarts_total > 3`) cause permanent firing loops once a counter exceeds the threshold. Our implementation uses `increase( ... [5m] ) > 0` , guaranteeing alerts only fire when an active restart occurs within the last 5 minutes.

3.3 Self-Healing Operator Engine (`operator-go/`)

Written in Go using the official `k8s.io/client-go` library:

- **Webhook Receiver:** Exposes `POST :8080/webhook` accepting JSON payloads from Alertmanager.

- **Anti-Flap Cooldown Engine:** Implemented via a thread-safe `sync.Map` cache. When an alert triggers remediation, a 60-second cooldown lock is placed on that alert key. Duplicate alerts arriving during ongoing recovery are safely ignored to avoid thrashing.
- **DNS Remediation & Backup Fallback:** Evicts crashed CoreDNS pods to force clean scheduling by the ReplicaSet. Concurrently deploys a fallback DNS service (`backup-dns`) utilizing public resolvers `8.8.8.8` and `1.1.1.1`) to restore cluster-wide resolution during upstream outages.
- **NetworkPolicy Safe Restoration:** Scans namespaces for blocking policies (e.g. `block-all-ingress`), purges the misconfigured policy, and applies a non-destructive `baseline-allow-all` policy.
- **Multi-CNI Remediation:** Automatically detects whether Calico (`calico-system`) or Flannel (`kube-flannel`) is active and evicts failing CNI daemonset pods to trigger fresh socket binding.

4. Fault Injection, Chaos Testing & Verification

To rigorously validate the self-healing loops, we developed a chaos injection script (`fault-injection/inject_faults.sh`) that runs automated assertions without human intervention.

4.1 Test Scenarios & Automated Verification Results

Test Scenario	Fault Injection Command	Observed Detection	Operator Action	Recovery Result
CoreDNS Crash	<code>./inject_faults.sh kill-coredns</code> (Force deletes CoreDNS pod)	<code>dns_healthy</code> drops to 0; Prometheus triggers <code>DNSResolutionFailed</code>	Operator receives webhook, evicts faulty pod, deploys <code>backup-dns</code>	PASS DNS resolved within 14s
NetworkPolicy Block	<code>./inject_faults.sh apply-bad-policy</code> (Zero-ingress blocking rule)	<code>connectivity_success</code> drops to 0; Prometheus triggers <code>PodConnectivityBlocked</code>	Operator identifies foreign policy, deletes block, restores baseline allow	PASS HTTP 200 restored within 18s
CNI Node Failure	<code>./inject_faults.sh kill-cni</code> (Kills Calico/Flannel node pod)	Node pod fails; Prometheus triggers <code>CNIPodCrash</code>	Operator triggers CNI daemonset pod restart	PASS Node Ready restored within 16s

4.2 Test Suite Execution Output

```

=== Running Full Test Scenario with Automated Assertions ===

Step 1: Baseline Health Check
>>> [TEST #1] Baseline DNS Resolution
>>> [PASS] Baseline DNS Resolution
>>> [TEST #2] Baseline Pod-to-Pod Connectivity
>>> [PASS] Baseline Pod-to-Pod Connectivity

Step 2: Injecting CoreDNS Crash Fault
Waiting for CoreDNS self-healing recovery (up to 30s) ...
CoreDNS recovered after ~ 8 seconds
>>> [TEST #3] CoreDNS Self-Healing and Resolution Recovery
>>> [PASS] CoreDNS Self-Healing and Resolution Recovery

Step 3: Injecting NetworkPolicy Blocking Fault
>>> [TEST #4] Verify Network Traffic is Blocked by Policy
>>> [PASS] Verify Network Traffic is Blocked by Policy
Waiting for NetworkPolicy self-healing remediation ...
NetworkPolicy remediated after ~ 12 seconds
>>> [TEST #5] Pod-to-Pod Traffic Restored after NetworkPolicy Remediation
>>> [PASS] Pod-to-Pod Traffic Restored after NetworkPolicy Remediation

Step 4: Injecting CNI Node Crash Fault
Waiting for CNI node self-healing recovery (up to 30s) ...
CNI node recovered after ~ 10 seconds (all 3 nodes Ready)
>>> [TEST #6] CNI Node Recovery and Cross-Node Connectivity
>>> [PASS] CNI Node Recovery and Cross-Node Connectivity

```

AUTOMATED TEST SCENARIO REPORT

```

Total Tests Executed: 6
Tests Passed:        6
Tests Failed:        0
=== ALL SCENARIOS PASSED SUCCESSFULLY ===

```

5. Observability & Monitoring Dashboard

A unified Grafana dashboard is declared in `manifests/monitoring/grafana-dashboards.yaml` and imported automatically via Grafana sidecar. The dashboard provides:

- **Instant Health Gauges:** Real-time DNS Latency, DNS Status indicator (Green/Red), Connectivity percentage, and CNI restart counters.
- **Time-Series Metrics:** Millisecond-level query latency graphs showing spikes during fault injection and rapid restoration post-remediation.
- **Integrated Loki Logs:** Real-time log streams filtering operator remediation actions across `monitoring`, `kube-system`, and `calico-system`.

6. Conclusion & Learning Outcomes

The project successfully achieves full-scope, production-grade automated network recovery on Kubernetes:

- **Limits of Vanilla Kubernetes Understood:** Identified why application-level self-healing fails during networking infrastructure degradation.
- **Custom Controller Engineering:** Built and validated a resilient Go operator utilizing `client-go` with robust cooldown logic to prevent cascading restart storms.
- **Full Observability Pipeline:** Engineered an end-to-end telemetry loop connecting custom Python probes $\rightarrow$ Prometheus ServiceMonitors $\rightarrow$ Alertmanager Webhooks $\rightarrow$ Go Operator.
- **Reproducible Chaos Verification:** Established automated chaos verification test suites guaranteeing verifiable reliability.